AIC 技术方案 · 2026

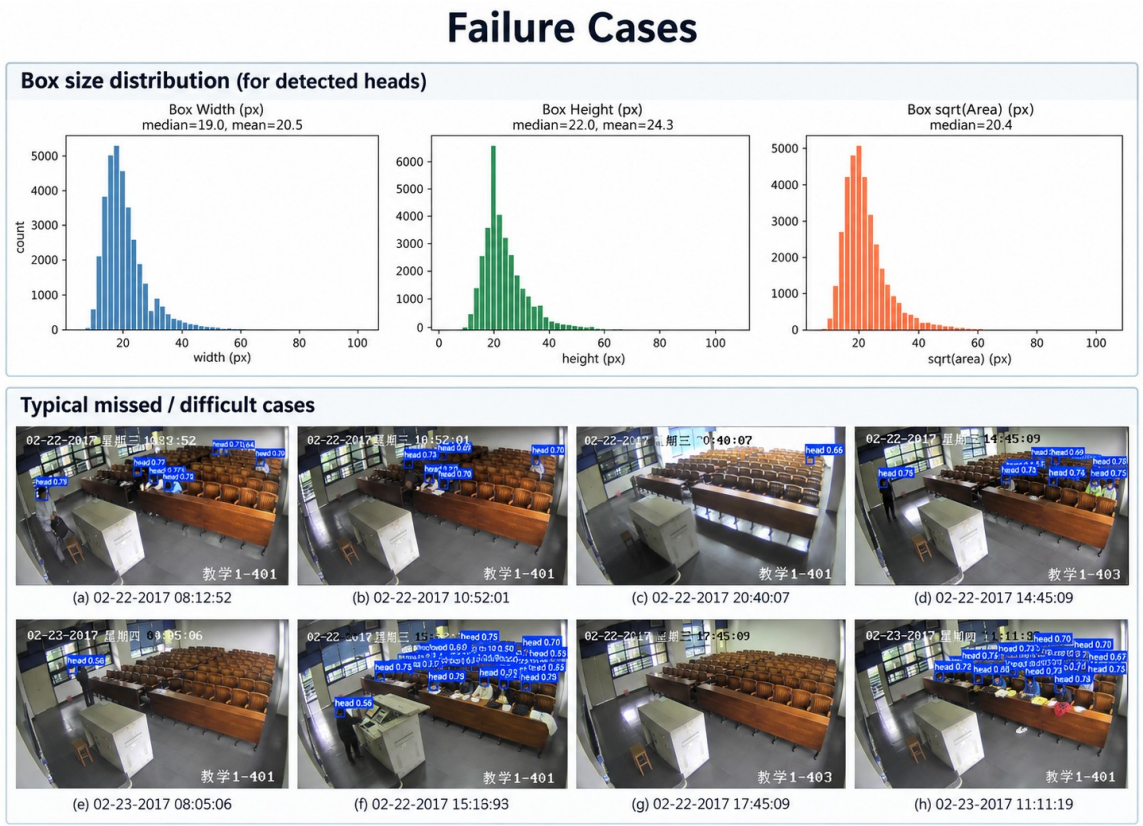
面向校园监控的轻量小目标检测

只改 YOLOv8n 的颈部：接入 P2 高分辨率特征 + 可学习加权融合；骨干与检测头不动

mAP50 **0.929 → 0.936** 参数 **2.93M**

同一 val 集 · 同一 imgsz=640 · FPS 同机测

后排学生只有十几像素，官方 n 基本漏掉



失败一页：官方 YOLOv8n 的漏检举例

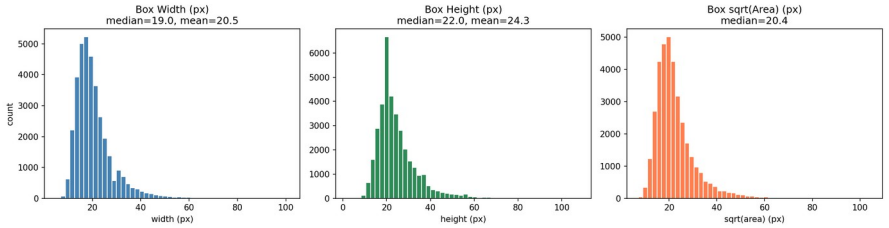
92.4% 的目标框宽度不到 32 像素

这是典型的小目标场景，不是拍摄质量问题。

- 后排约 20 像素的头，在最小画框层上只占 1 ~ 2 个格子
- 位置信息几乎被缩没 → 会漏，框也会飘
- 把模型换大能改善一部分，但参数量和计算量一起涨，边缘设备跑不动

根因：最小步长是 8，而小头还在步长 4 那一层

层	步长	640 输入时特征图边长	约 20 像素的头占几格
P2	4	160 × 160	约 5 格
P3	8	80 × 80	约 1 ~ 2 格
P4	16	40 × 40	不到 1 格
P5	32	20 × 20	几乎没有



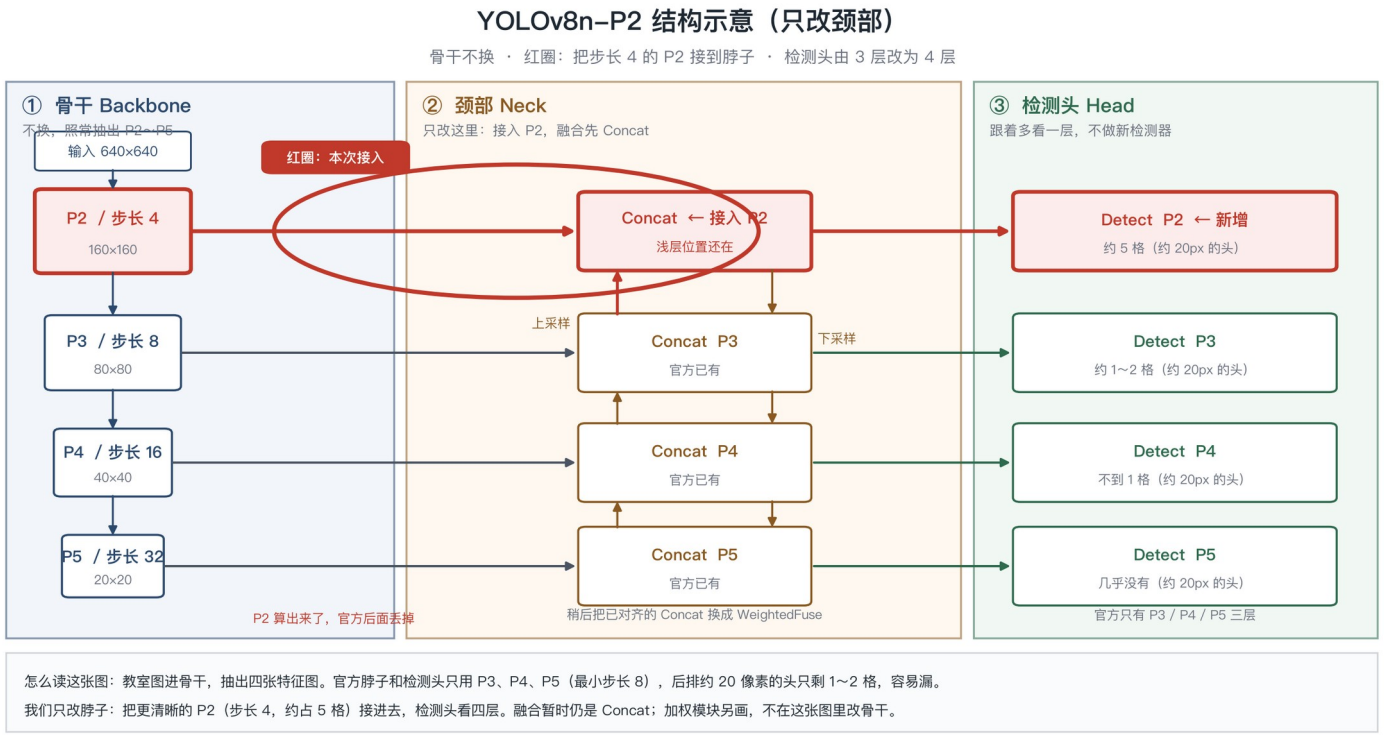
目标框尺寸分布（宽 / 高 / 对角线，单位 px）

- 漏检首先来自步长太大、融合太晚，不是骨干不够深
- 所以结论是：只改颈部

官方检测头只接 **P3/P4/P5**：小头还在的那一层，没参与画框。

- 更浅、更清晰的 P2（步长 4）骨干已经算出来了
- 融合只做 Concat：浅层管位置、深层管类别，没有按目标大小加权

只改颈部两处，骨干和检测头不动



怎么读这张图：教室图进骨干，抽出四张特征图。官方脖子和检测头只用 P3、P4、P5（最小步长 8），后排约 20 像素的头只剩 1~2 格，容易漏。

我们只改脖子：把更清晰的 P2（步长 4，约占 5 格）接进去，检测头看四层。融合暂时仍是 Concat；加权模块另画，不在这张图里改骨干。

改动一：接入 P2

检测头从三个尺度改为四个尺度，最小检测步长 8 → 4。配置沿用官方 yolov8-p2.yaml ，不手搓层号。

改动二： Concat → 加权融合

特征图对齐之后，把通道拼接换成可学习加权求和，只替换融合层，不改 Conv / C2f 。

图：骨干 → 颈部（红圈为接入 P2 ）→ 检测头

加权融合：让每层按目标大小贡献

$$F = \sum w_i X_i, \quad w_i \geq 0, \quad \sum w_i = 1$$

X_i

已经对齐、通道数相同的输入特征图

w_i

可学习权重，训练时用 softmax 保证非负且和为 1

实现

yaml 仍写标准 Concat 让 ultralytics 建图，训练前替换为加权模块

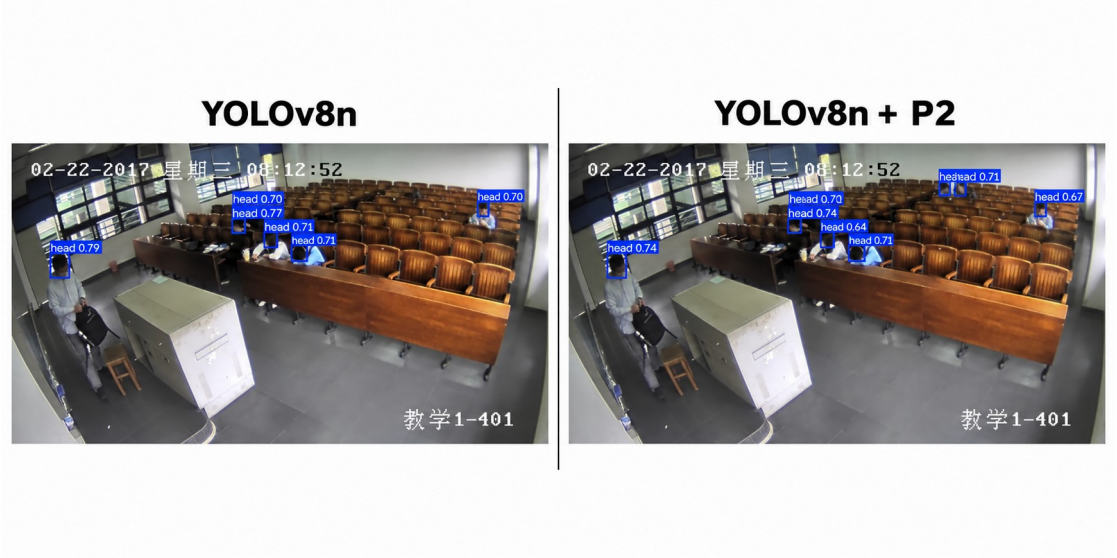
只改融合，不改 **Conv / C2f**，不换骨干。

教室数据：接上 P2 拿到主要收益，参数反而略降

方法	mAP50	mAP50-95	参数	FLOPs	FPS
yolov8n（官方）	0.929	0.416	3.0 M	8.2 G	233.14
+ P2（主要改动）	0.936	0.425	2.93 M	12.2 G	199.01
+ 加权融合（完整方法）	0.933	0.421	3.31 M	13.9 G	156.77
对比：yolov8s	0.940	0.433	11.13 M	28.4 G	228.51
对比：yolov5nu	0.921	0.412	2.50 M	7.1 G	226.87

- 接 P2：mAP50 0.929 → 0.936，参数量 3.0 M → 2.93 M（略降），代价是 FPS 233 → 199
- 再加权：0.933，没有超过只接 P2，FPS 199 → 157 —— 我们如实报告，不当成增益
- 不用 yolov8s：它 mAP50 0.940，但要 11.13 M 参数、28.4 G FLOPs（约为 P2 方案的 3.8 倍）

同一张图：官方漏的，接上 P2 就框住了



左：官方 yolov8n ｜ 右：接入 P2 后



n ｜ P2 ｜ 完整方法（三列对比）

远景场景（TinyPerson）：如实报告，没有涨点

方法	mAP50	mAP50-95	参数
yolov8n（官方）	0.127	0.0367	3.0 M
完整方法（P2 + 加权融合）	0.124	0.0363	3.31 M

- 同一套配置、同样 100 epoch、同样 imgsz=640，只换数据集
- TinyPerson 目标更小更密集，是公开数据里最极端的一档
- 结果基本持平（-0.003），我们把它写成方法边界，不当成绩

这说明什么

极端远景小目标，对 3M 量级的轻量模型是超纲的：只改颈部不足以解决。

要上岗还得用本校远端监控画面重新标数据、重新微调 —— 这一步我们没有做，也不宣称做了。

教室数据上的提升，才是本方案的主要证据。

三个必须说清楚的地方

主要贡献是 P2，不是加权

接 P2 带来 +0.007 mAP50；加权融合为 0.933，未超过 P2。答辩主讲 P2。

FPS 有口径

measure_speed.py 用构造图、单张循环；yolo val 的真实吞吐含 NMS，约为前者的 1/2 ~ 1/3。两者不可直接比较。

未跨机型验证

所有 FPS 在同一台 RTX 5090 D 上测；换机器复现时 mAP 稳定，FPS 会有 ±5% 波动。

陌生人按 README 能跑通

复现 mAP50（官方 n）	复现 FPS	权重参数量
0.931	224.4	3.0 M

- 9/29 在另一台机器换目录按 README 走通 val，数字与 csv 一致（版本差异 ±0.002）
- 仓库含五个教室权重、两个 TinyPerson 权重的说明与数据链接
- fuse 推理必须用 scripts/predict_fuse.py，否则加权层会退化成普通 Concat

可复现 · 可交付 · 数字可追溯（全部来自 docs/ 实验记录 .csv）